

Simulated Annealing aplicado a la Función de Ackley: `SimulatedAnnealingAckley.m`

Autor: Prof. Dr. Aboud Barsekh Onji

Institución: Facultad de Ingeniería, Universidad Anáhuac México

Contacto: aboud.barsekh@anahuac.mx

ORCID: 0009-0004-5440-8092

Introducción

Este script implementa el algoritmo **Simulated Annealing (SA)** — también conocido como *Recocido Simulado* — aplicado a la optimización de la **función de Ackley** en dos dimensiones. El ejemplo es ideal como caso de estudio porque la función de Ackley es deliberadamente difícil: su superficie contiene cientos de mínimos locales que atrapan a los optimizadores clásicos, pero tiene un único mínimo global en el origen $f(0, 0) = 0$.

El script cubre el ciclo completo de trabajo: definición de la función objetivo, configuración de parámetros del algoritmo, ejecución del lazo principal con visualización en tiempo real de la trayectoria explorada, y análisis de convergencia.

1. Función Objetivo: Ackley

La función de Ackley es uno de los **benchmarks estándar** en optimización metaheurística. Su definición matemática es:

$$f(x, y) = -20 \exp\left(-0.2 \sqrt{0.5(x^2 + y^2)}\right) - \exp(0.5(\cos 2\pi x + \cos 2\pi y)) + 20 + e$$

Propiedades clave

Propiedad	Valor
Mínimo global	$f(0, 0) = 0$
Dominio típico	$[-5, 5]^2$
Tipo de superficie	Multimodal (muchos mínimos locales)
Número de variables	2 (extensible a n)
Parámetros estándar	$a = 20, b = 0.2, c = 2\pi$

La superficie tiene una zona central casi plana rodeada por un “muro” de valles cosénicos. Un algoritmo que solo desciende localmente (como el gradiente) se queda atrapado en cualquiera de estos valles antes de llegar al origen.

Implementación en MATLAB

```
ackley = @(x) -20 * exp(-0.2 * sqrt(0.5 * (x(1)^2 + x(2)^2))) ...  
             - exp(0.5 * (cos(2 * pi * x(1)) + cos(2 * pi * x(2)))) ...  
             + 20 + exp(1);
```

Se define como una **función anónima** (`@(x)`) que recibe un vector de dos elementos `[x(1), x(2)]`. Esto permite evaluar la función con una sintaxis compacta: `ackley([0, 0])` devuelve 0.

La malla de visualización se genera con `meshgrid` y se evalúa con `arrayfun`, que aplica la función elemento a elemento:

```
[x_grid, y_grid] = meshgrid(-5:0.1:5, -5:0.1:5);  
z_grid = arrayfun(@(x, y) ackley([x, y]), x_grid, y_grid);
```

¿Por qué `arrayfun`? La función anónima recibe un vector, no opera de forma vectorizada directamente sobre matrices. `arrayfun` resuelve esto sin necesidad de bucles explícitos.

2. Parámetros del Algoritmo

La calidad de los resultados del SA depende críticamente de la elección de sus parámetros. El script utiliza los siguientes valores:

```
T          = 1000;      % Temperatura inicial  
T_min      = 1;         % Temperatura mínima (criterio de paro)  
alpha      = 0.95;      % Factor de enfriamiento geométrico  
max_iter   = 700;       % Número máximo de iteraciones  
x          = [0.1, 0.1]; % Solución inicial
```

¿Cómo se relacionan los parámetros?

Parámetro	Símbolo	Efecto al aumentar su valor
Temperatura inicial	T_0	Mayor exploración al inicio; mayor riesgo de tardanza
Temperatura mínima	$T_{\min}$	Mayor $T_{\min}$ termina antes; menor refinamiento
Factor de enfriamiento	α	Más cercano a 1 $\rightarrow$ enfriamiento más lento $\rightarrow$ mejor calidad

Iteraciones máximas	$N_{\max}$	Más iteraciones → más evaluaciones → mayor cómputo
---------------------	------------	--

Cálculo del número efectivo de pasos de temperatura:

$$N_T = \left\lceil \frac{\ln(T_{\min}/T_0)}{\ln(\alpha)} \right\rceil = \left\lceil \frac{\ln(1/1000)}{\ln(0.95)} \right\rceil \approx 135 \text{ pasos}$$

Con `max_iter = 700`, el enfriamiento termina antes de agotar las iteraciones (el criterio $T < T_{\min}$ se activa primero).

3. Inicialización y Estructuras de Rastreo

```
best_sol = x;
best_cost = ackley(x);

trace_x = x(1);
trace_y = x(2);
trace_z = best_cost;
costs = zeros(1, max_iter);
```

Se mantienen **dos tipos de seguimiento**:

- **best_sol / best_cost**: la mejor solución encontrada *en toda la historia* del algoritmo (no se modifica si el SA acepta una solución peor).
 - **trace_x, trace_y, trace_z**: la trayectoria de la solución *actual* (incluye movimientos a soluciones peores cuando el criterio de Metrópolis lo acepta). Sirve para visualizar la exploración.
 - **costs**: guarda el mejor costo en cada iteración (para la gráfica de convergencia).
-

4. Lazo Principal: Criterio de Aceptación de Metrópolis

Esta es la parte central del algoritmo. En cada iteración ocurren tres pasos:

Paso 1 — Generar un vecino

```
new_x = x + randn(1, 2);
new_cost = ackley(new_x);
```

La nueva solución candidata se genera perturbando la solución actual con ruido gaussiano estándar $\mathcal{N}(0, 1)$. Esto produce un **movimiento aleatorio** en el espacio de búsqueda cuya magnitud es aleatoria.

Nota pedagógica: La magnitud del paso no se escala con T , lo que es una simplificación. En implementaciones más sofisticadas, el tamaño del paso también se reduce con la temperatura.

Paso 2 — Criterio de aceptación

```

if new_cost < best_cost
    best_sol = new_x;
    best_cost = new_cost;
    x = new_x;
else
    delta_cost = new_cost - ackley(x);
    if rand < exp(-delta_cost / T)
        x = new_x;
    end
end
end

```

El **criterio de Metrópolis** determina si se acepta la nueva solución:

- Si $f(x_{\text{nuevo}}) < f(x_{\text{mejor}})$: se acepta **incondicionalmente** y se actualiza el mejor global.
- Si $f(x_{\text{nuevo}}) \geq f(x_{\text{actual}})$: se acepta con **probabilidad**:

$$P = \exp\left(\frac{-\Delta E}{T}\right), \quad \Delta E = f(x_{\text{nuevo}}) - f(x_{\text{actual}})$$

Esta probabilidad disminuye conforme aumenta ΔE (solución mucho peor) o disminuye T (sistema frío).

Situación	Probabilidad de aceptar
T alto, ΔE pequeño	≈ 1 (casi seguro acepta)
T alto, ΔE grande	Moderada
T bajo, ΔE pequeño	Baja
T bajo, ΔE grande	≈ 0 (casi nunca acepta)

Paso 3 — Actualizar temperatura y rastreo

```

costs(iter) = best_cost;
trace_x(end+1) = x(1);
trace_y(end+1) = x(2);
trace_z(end+1) = ackley(x);
T = T * alpha;
if T < T_min
    break;
end
end

```

Superficie 3D de la función de Ackley con colormap jet

Figure 1: Superficie 3D de la función de Ackley con colormap jet

Trayectoria explorada por el SA sobre la superficie de Ackley

Figure 2: Trayectoria explorada por el SA sobre la superficie de Ackley

La temperatura se reduce geométricamente: $T_{k+1} = \alpha \cdot T_k$. Cuando $T < T_{\min}$, el bucle termina anticipadamente con **break**.

5. Visualización

El script genera **tres** figuras:

Figura 1 — Superficie de la función de Ackley

```
surf(x_grid, y_grid, z_grid, 'EdgeColor', 'interp');  
colormap jet;
```

Muestra la topografía de la función: la zona azul central (mínimo global) y las crestas de colores que representan los mínimos locales. Se usa 'EdgeColor', 'interp' para una visualización suave.

Figura 2 — Trayectoria del algoritmo en tiempo real

```
scatter3(trace_x, trace_y, trace_z, 50, 'r', 'filled');  
drawnow;
```

En cada iteración se añaden puntos rojos que muestran los puntos visitados por el SA. Al inicio (alta T) los puntos están dispersos por toda la superficie; al final (baja T) se concentran cerca del mínimo. **drawnow** fuerza la actualización gráfica en cada iteración.

Figura 3 — Convergencia del costo

```
plot(1:iter, costs(1:iter), 'LineWidth', 2);
```

Muestra la evolución del **mejor costo encontrado** por iteración. La curva descende con saltos irregulares al inicio y se estabiliza cerca de cero al converger.

Observación importante: La curva de **costs** no es monótonamente decreciente en la trayectoria actual (**trace_z**), pero sí lo es si

Curva de convergencia del mejor costo en el SA

Figure 3: Curva de convergencia del mejor costo en el SA

se grafica `best_cost` en cada iteración. En este script, `costs(iter) = best_cost` garantiza que la curva de convergencia sea monótonamente no creciente.

6. Resultados Esperados

Al ejecutar el script con los parámetros predeterminados, la salida en consola debe ser aproximadamente:

```
Mejor solución encontrada: (0.0018, -0.0031)
Costo de la mejor solución: 0.0089
```

Los valores exactos varían entre ejecuciones por la naturaleza estocástica del algoritmo. En la mayoría de las ejecuciones, el SA encuentra soluciones con $f < 0.1$, muy cercanas al óptimo global $f(0,0) = 0$.

7. Tabla de Conceptos Clave

Concepto	Definición	Rol en este código
Función anónima	Función definida con <code>@(args)</code> sin nombre propio	Define <code>ackley(x)</code> de forma compacta
Criterio de Metrópolis	Regla probabilística de aceptación de soluciones peores	<code>rand < exp(-delta_cost / T)</code>
Temperatura T	Parámetro de control que regula la probabilidad de aceptación	Decrece con <code>T = T * alpha</code>
Factor de enfriamiento α	Constante en $(0,1)$ que reduce T en cada iteración	<code>alpha = 0.95</code>
drawnow	Función MATLAB que fuerza la actualización de figuras	Visualización en tiempo real
randn(1,2)	Vector de 2 números gaussianos $\mathcal{N}(0,1)$	Perturbación aleatoria del vecino
arrayfun	Aplica una función elemento a elemento sobre arrays	Evalúa <code>ackley</code> sobre la malla <code>meshgrid</code>
break	Termina el bucle anticipadamente	Criterio de paro térmico <code>T < T_min</code>

8. Posibles Modificaciones para Experimentar

1. **Cambiar α :** prueba con `alpha = 0.99` (enfriamiento lento) vs `alpha = 0.80` (rápido). Observa cómo cambia la calidad de la solución y la velocidad de convergencia.
2. **Escalar el paso con T :** modifica la perturbación a `new_x = x + randn(1,2) * (T / T_0)` para que los saltos sean más grandes cuando la temperatura es alta.
3. **Extender a 3D:** cambia la definición de `ackley` para que acepte un vector de n variables y experimenta con dimensiones superiores.
4. **Múltiples corridas:** ejecuta el SA $N = 30$ veces y analiza estadísticamente la distribución de la mejor solución encontrada (media, desviación estándar, boxplot).

Las imágenes se generan al ejecutar el script en MATLAB. Se recomienda guardarlas con `saveas(gcf, 'SA_surface.png')` después de cada `figure`.